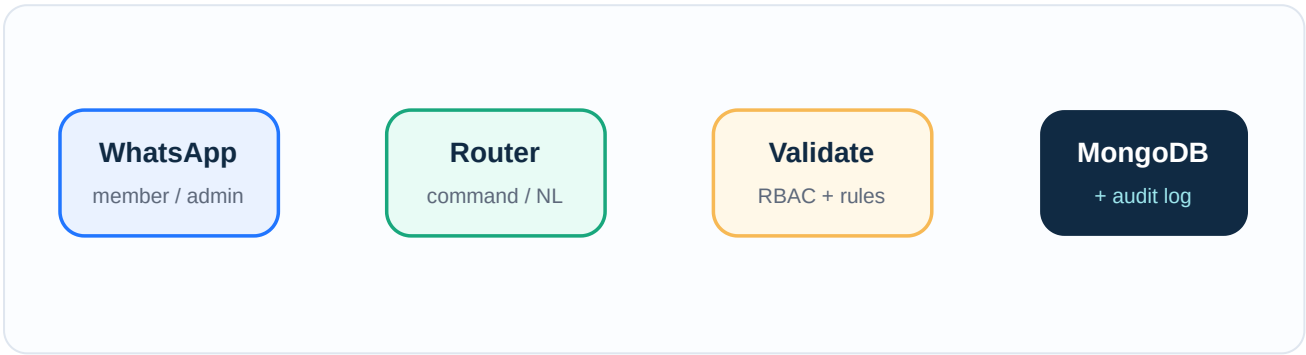
PBBot

A WhatsApp-first community operations platform for events, tasks, updates, reminders, and reports.



Guiding idea: every action is a named operation. Natural language can suggest operations, but only validated operations can change system state.

01 · EXECUTIVE VIEW

What we are building

PBBot helps a community run programs and activities through the channel members already use: WhatsApp. Admins configure the system; members see assigned work and submit progress; the system keeps the database and audit log authoritative.

> **Product promise:** less manual follow-up for admins, clearer ownership for members, and reliable history for the team.

Goals

Operational clarity

One reliable view of members, labels, events, tasks, assignments, updates, and status.

Deterministic execution

Every action is validated, role-checked, executed by business logic, and audited.

WhatsApp-first access

Members can participate without learning a separate app.

Extensible foundation

Dynamic schemas support changing participation programs without code changes to every form.

What v1 is not

It is not a general CRM, a replacement for WhatsApp, a native mobile app, or an autonomous AI agent with database access. Natural language is optional and bounded by the operation layer.

02 · WHO IT SERVES

Users and permissions

Member

Read own work | Submit updates | View own history

- View assigned events and tasks
- Submit and edit own updates
- View permitted status and history

Cannot create, assign, delete, or administer records.

Admin

Configure | Assign | Report

- Manage users and labels
- Manage events, schemas, and tasks
- Configure reminders and reports
- View audit history and system configuration

Security rule: WhatsApp commands are an interface, not an authorization mechanism. The backend must authorize every operation server-side.

Core data model

Entity	Role in the product	Key links
User	Community member or admin.	Labels, assignments, audit actor.
Label	Reusable classification such as Backend, Core Team, or Fourth Year.	Many-to-many with users and events.
Event	Top-level program or activity.	Participation schema or organization tasks.
Assignment	Explicit connection between a user and event/task.	Status, reminders, missed count, last update.
Update	Progress record against an assignment field.	Assignment, actor, schema validation.
Audit log	Immutable operation history.	Actor, operation, payload, result, timestamp.

03 · PRODUCT WORKFLOWS

How the system is used

1. **Member checks work.**
   Member sends `/my`, `/events`, or `/tasks`; the bot returns active assignments, statuses, and due dates.
2. **Member submits progress.**
   Member uses `/update`; the system confirms assignment ownership, field type, required values, and then records the update.
3. **Admin configures a program.**
   Admin creates an event, attaches a dynamic participation schema or creates organization tasks, and assigns members directly or by label.
4. **System follows up.**
   A scheduled reminder run finds eligible assignments, sends WhatsApp DMs, records delivery results, increments missed counts, and escalates after a configured threshold.
5. **Admin reviews outcomes.**
   Progress, pending, completed, and audit reports are generated from persisted records.

Two event modes

Participation event

Collect structured information such as organization, PR count, PR links, proposal submitted, mentor, and accepted status.

GSoC LFX Research

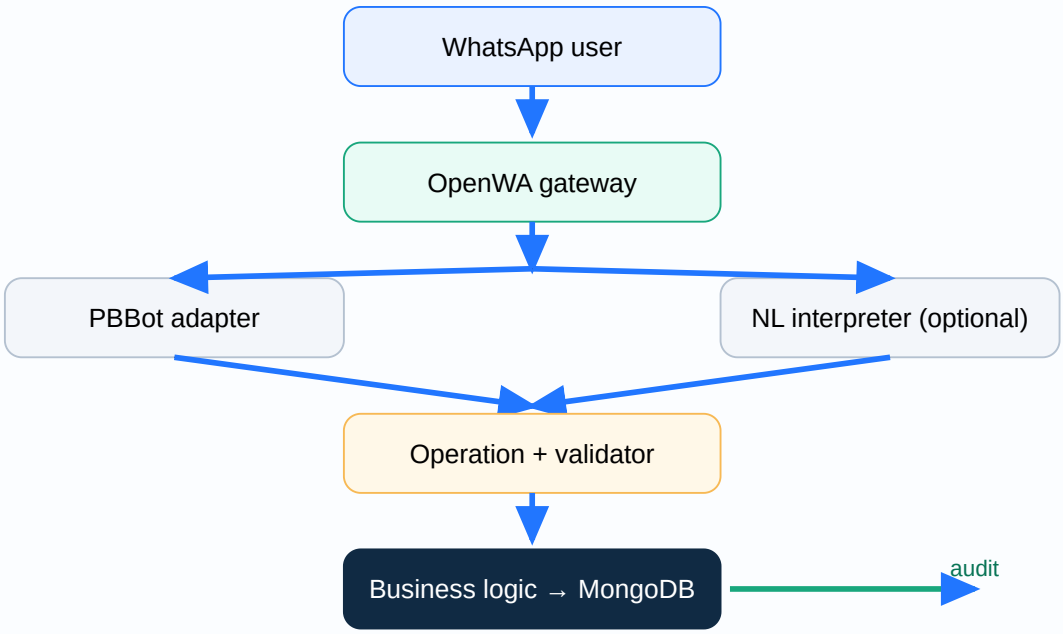
Organization event

Coordinate work through tasks with name, description, assignee, due date, status, and priority.

Recruitment Hackathon Workshop

04 · PRODUCT ARCHITECTURE

Safe path from message to state



Non-negotiable boundary: the LLM may translate a message into proposed operations. It may not query repositories, execute commands, or mutate MongoDB directly.

Operation envelope

```
{ "name": "task.create", "payload": { ... }, "actorId": "user-id", "source": "command" }
```

OpenWA owns WhatsApp connectivity, sessions, and message delivery. PBBot consumes its authenticated webhook/API contract and owns normalization, actor mapping, operations, RBAC, MongoDB, and domain audit. Validation order: envelope → operation name → payload → role → referenced records → business rules → execution → audit.

Flow	OpenWA does	PBBot does
Inbound	Receives WhatsApp message and emits signed webhook.	Verifies, deduplicates, normalizes, identifies actor, routes.
Outbound	Sends through the configured session and returns provider status.	Renders replies/reminders and records delivery/audit results.
Session	QR, authentication, reconnect, session health.	Checks readiness and reports actionable failures.

05 · FUNCTIONAL SCOPE

What the MVP must support

Area	Required capabilities	Primary users
Users & labels	Create, update, deactivate, list users; create/update/delete/list labels; assign/remove labels.	Admin
Events	Create, edit, soft-delete, list, assign/unassign, and change status.	Admin
Schemas	Dynamic fields: text, number, boolean, date, URL, single select, multi select, list.	Admin
Tasks	Create, edit, delete, assign, complete, list; due dates and priority.	Admin
Updates	Submit, edit, and view history; validate against ownership and schema.	Member / Admin
Reminders	Configure frequency and escalation; run idempotently; record delivery history.	Admin / System
Reports	Progress, pending, completed, and filtered reports.	Admin
Audit	Record successful and rejected operations; list and filter immutably.	Admin

WhatsApp commands

Member

`/help` `/my` `/events` `/tasks` `/update` `/history` `/status`

Admin

`/users` `/labels` `/events` `/schema` `/tasks` `/assign` `/reminders` `/reports` `/audit`

Operation catalogue

The complete operation list is defined in the source PRD. The implementation must preserve stable names and payload contracts so that both commands and natural language target the same business layer.

`user.*` · `label.*` · `event.*` · `schema.*` · `task.*` · `update.*` · `reminder.*` · `report.*` · `audit.*`

06 · QUALITY BAR

Rules that keep the product trustworthy

Security

- Server-side RBAC for every operation
- Webhook verification and secret management
- Redacted sensitive audit payloads

Reliability

- Idempotent reminder runs
- Delivery failures are visible
- Soft deletion preserves history

Performance

- Typical command response under 2 seconds, excluding provider latency
- Pagination/bounds on lists and reports
- Indexes for common filters

Observability

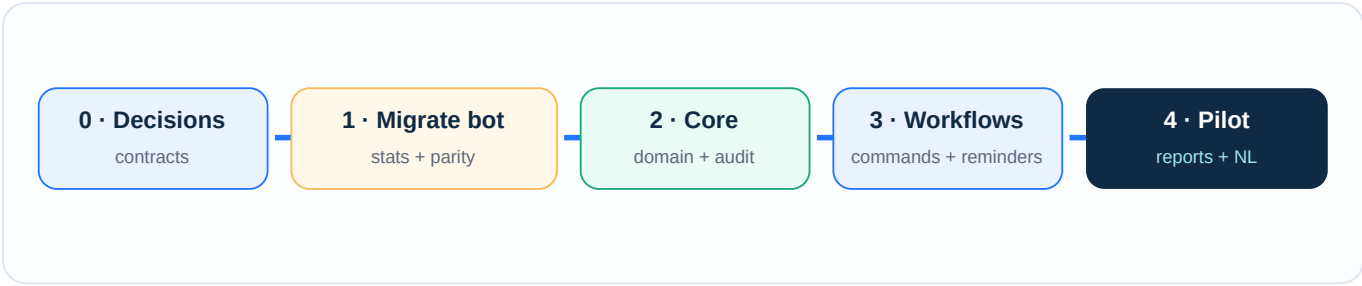
- Correlation ID, operation, actor, duration, outcome
- No message bodies or secrets in default logs
- Application and database health checks

Definition of MVP-ready

1. The existing `!stats` response and nightly configured-group broadcast work through OpenWA and the PBBot operation/audit path with parity tests.
2. A member can be identified by WhatsApp ID, view assignments, and submit a validated update.
3. An admin can configure an event, schema/tasks, and assignments.
4. Reminder runs find pending work, record delivery outcomes, and escalate correctly.
5. Progress, pending, completed, and audit reports are available to admins.
6. Tests cover role restrictions, schema validation, assignment ownership, reminder idempotency, duplicate delivery, and migration parity.
7. The service runs locally and in a container with documented environment variables.

07 · WORK PACKAGES

Build in reviewable slices



Work package	Outcome	Exit check
0 · Decisions	Finalize data model, operation payloads, OpenWA session/API contract, statuses, and pilot event.	Team approves contracts.
1 · Existing bot migration	Move `!stats`, nightly broadcasts, `GROUP_ID` / `GROUP_IDS`, optional stickers, and MongoDB stats reads behind OpenWA and PBBot operations.	Behavior parity, idempotency, delivery tracking, and failure tests pass.
2 · Core domain	Users, labels, events, schemas, tasks, assignments, updates, RBAC, audit.	API tests pass against MongoDB.
3 · Workflows	Webhook normalization, command router, guided flows, outbound messaging, reminders, escalation, and history.	End-to-end member/admin workflow completes through OpenWA.
4 · Pilot	Reports, safe natural language, monitoring, deployment, and one real event.	Admins can operate without manual spreadsheet backup.

08 · EXISTING FOUNDATIONS

What we can build on

> **Reuse infrastructure patterns, not the domain model.** OpenWA and media_automata are substantially more relevant internal foundations than the small supplied stats-bot reference.

Foundation	Already provides	PBBot use
OpenWA NestJS / TypeScript	Sessions, QR lifecycle, REST, HMAC webhooks, API keys, rate limits, queues, health checks, audit logs, Docker, pluggable DB/storage/cache.	Use as the WhatsApp transport gateway behind a PBBot adapter.
media_automata Python / FastAPI	Webhook normalization, command orchestration, typed LLM intent parsing, jobs/tasks, retries, heartbeats, idempotency, artifacts, deployment checks, tests.	Reuse reliability patterns; keep PBBot's operation contract as the boundary.
whatsapp-bot Node.js reference	Trigger words, MongoDB reads, scheduled broadcasts, environment configuration.	Keep as a simple reference for message handlers and cron behavior.

Concrete media_automata examples to take as reference

PBBot step	media_automata precedent	How PBBot applies it
Normalize inbound OpenWA events	`whatsapp/normalizer.py:44` — `normalize_openwa_payload()`	Normalize once into a stable message object before actor lookup or command parsing.
Send replies/reminders	`whatsapp/client.py:61` — `OpenWAClient.send_text()`	Use an OpenWA client interface and persist returned delivery IDs/status.
Route messages	`orchestrator.py:124` — `process_whatsapp_message()`	Use one orchestration entry point that hands off only typed PBBot operations.
LLM intent parsing	`agents/graph.py:139` — `SocialAgentGraph.run()`	Generate proposed operations, then validate and confirm them before execution.
Idempotency	`repository.py:create_job()` plus `tests/test_repository.py`	Use OpenWA event/message ID to prevent duplicate operation execution.
Retries and worker recovery	`repository.py:347`, `worker.py:274`, `retry.py`	Atomically claim reminder/report jobs, heartbeat them, retry transient failures, and recover stale work.
Scheduling	`scheduling.py:79` plus `tests/test_scheduling.py`	Centralize timezone-aware reminder parsing and explicit command precedence.
Operational health	`api.py:45` and `monitoring.py:236`	Expose separate PBBot, OpenWA, session, queue, and stale-work checks.

Boundary reminder: PBBot-specific users, labels, events, schemas, assignments, updates, reminders, reports, and domain audit remain new work. The LLM may propose PBBot operations, but never calls either repository or database directly.

09 · DECISIONS BEFORE CODING

Questions for the team

OpenWA ownership

Which OpenWA session/phone number will own the bot, and where will OpenWA be deployed?

Integration contract

Which OpenWA webhook events, API endpoints, signature rules, and delivery acknowledgements will PBBot support?

Pilot

Will the first real workflow be a participation event or an organization event?

Status model

What exact status values and transition rules apply to events, tasks, and assignments?

Reminders

What timezone, sending window, escalation threshold, and admin destination should be used?

Natural language

Should NL be deferred until command workflows are stable, or included behind confirmation in the MVP?

Recommended team decision: approve the operation boundary and run one pilot event end-to-end before expanding the natural-language layer.

10 · REFERENCES

Source material

1. **Supplied reference repository:** github.com/ShreyashSri/whatsapp-bot — WhatsApp bot for PBCTF statistics, with MongoDB, scheduled broadcasts, trigger words, Docker/PM2, and deployment configuration.
2. **Internal foundation — OpenWA:** `/home/unichronic/OpenWA`, especially `docs/03-system-architecture.md`, `docs/04-security-design.md`, `docs/05-database-design.md`, and `docs/06-api-specification.md`.
3. **Internal foundation — media_automata:** `/home/unichronic/media_automata`, especially the WhatsApp normalizer/client, command orchestrator, repository, schemas, scheduling, retry, monitoring, and tests.
4. **Product requirement:** PBBot — Product Requirements Specification (PRS) v2.0, reproduced and refined in the project workspace PRD.
5. **Product principle:** database is the source of truth; operations are validated and audited; natural language is an adapter to operations, not a database actor.